

Chương 2. Giao thức HTTP

- 2.1. Các phương thức HTTP phổ biến
- 2.2. Mã trạng thái HTTP phổ biến
- 2.3. Webservice API
- 2.4. Web client và các lỗi hỏng bảo mật Web
- 2.5. HTTPS, chứng chỉ SSL và quy trình hoạt động HTTPS/SSL

- **HTTP (HyperText Transfer Protocol):** HTTP là giao thức nền tảng của World Wide Web, cho phép các ứng dụng giao tiếp với nhau qua internet.
- **Vai trò của các phương thức HTTP:**
 - ✓ HTTP có nhiều phương thức để chỉ định hành động mong muốn với một tài nguyên trên máy chủ.
 - ✓ Các phương thức phổ biến nhất là GET, POST, PUT, DELETE thường được gọi là CRUD (Create, Read, Update, Delete).

- GET được dùng để yêu cầu lấy dữ liệu từ máy chủ mà không làm thay đổi dữ liệu.
- GET là phương thức **không có tác dụng phụ** (*idempotent*), nghĩa là nếu gọi nhiều lần thì vẫn cho ra cùng kết quả.
- Ví dụ thực tế: Trang hiển thị thông tin người dùng, kết quả tìm kiếm.

- HTTP Status Codes thường gặp: 200 (OK), 404 (Not Found).
- Lưu ý: GET không nên sử dụng để gửi dữ liệu nhạy cảm vì dữ liệu có thể xuất hiện trong URL

Phương thức POST

- POST được dùng để gửi dữ liệu mới đến máy chủ và có thể tạo một tài nguyên mới hoặc xử lý một yêu cầu đặc biệt.
- Dữ liệu được truyền trong phần body, thường là JSON hoặc XML
- Ví dụ: Tạo tài khoản mới, gửi biểu mẫu liên hệ.

Phương thức POST

- HTTP Status Codes thường gặp: 201 (Created), 204 (No Content).
- Lưu ý: POST có thể sử dụng để truyền tải dữ liệu nhạy cảm, vì dữ liệu được gửi trong phần body của yêu cầu và không hiển thị trên URL.

- PUT được dùng để cập nhật hoặc thay thế một tài nguyên có sẵn.
- PUT cũng là phương thức không có tác dụng phụ (*idempotent*), nghĩa là kết quả không thay đổi nếu gọi nhiều lần với cùng dữ liệu

- Ví dụ: Cập nhật thông tin người dùng, chỉnh sửa bài viết.
- HTTP Status Codes thường gặp: 200 (OK), 204 (No Content), 404 (Not Found).
- Lưu ý: PUT thường được sử dụng cho cập nhật toàn bộ tài nguyên. Khi chỉ cập nhật một phần, PATCH có thể được ưu tiên hơn

Phương thức DELETE

- DELETE dùng để xóa tài nguyên khỏi máy chủ.
- Ví dụ: Xóa tài khoản, xóa một bài viết.
- Nó thường trả về mã trạng thái 200 (OK), 204 (No Content), 404 (Not Found).
- **Lưu ý:** Phương thức này nên được sử dụng cẩn trọng để tránh xóa dữ liệu không mong muốn

- GET: Lấy dữ liệu
- POST: Tạo dữ liệu
- PUT: Cập nhật dữ liệu
- DELETE: Xóa dữ liệu

- Là mã số do máy chủ trả về để phản hồi các yêu cầu của client.
- Cấu trúc của mã trạng thái:
 - ✓ 3 chữ số: Con số đầu tiên biểu thị loại mã (2, 3, 4, 5), 2 chữ số tiếp theo chỉ định chi tiết mã phản hồi.
 - ✓ Ví dụ: 404 – “Không tìm thấy tài nguyên” hoặc 200 – “Yêu cầu thành công”.
- Mục đích: Giúp người phát triển ứng dụng hiểu rõ tình trạng của yêu cầu, từ đó cải thiện và xử lý ứng dụng tốt hơn.

- Mã trạng thái 2xx cho biết rằng yêu cầu từ client đã được máy chủ xử lý thành công.
- Các mã trạng thái phổ biến:
 - ✓ 200 OK: Yêu cầu thành công, tài nguyên được trả về như mong đợi. Ví dụ: Lấy dữ liệu người dùng qua GET /api/users.
 - ✓ 201 Created: Yêu cầu thành công và tài nguyên mới đã được tạo. Ví dụ: Tạo tài khoản mới bằng POST /api/users.
 - ✓ 204 No Content: Yêu cầu thành công nhưng không có nội dung để trả về (thường dùng cho DELETE hoặc PUT).

Thực hiện một yêu cầu GET đến một API công khai và xem mã trạng thái trả về (dùng công cụ như Postman).

Nhóm mã trạng thái 3xx – Điều Hướng (Redirection)

- Các mã trạng thái 3xx chỉ định rằng yêu cầu cần được chuyển hướng đến một URL khác để hoàn thành.
- Các mã trạng thái phổ biến:
 - ✓ 301 Moved Permanently: Tài nguyên đã được chuyển vĩnh viễn sang URL mới. Ví dụ: Địa chỉ trang web cũ chuyển hướng vĩnh viễn đến địa chỉ mới.

Nhóm mã trạng thái 3xx – Điều Hướng (Redirection)

- Các mã trạng thái phổ biến:
 - ✓ 302 Found: Tài nguyên tạm thời chuyển sang một URL khác. Ví dụ: Tạm thời chuyển hướng người dùng đến trang khuyến mãi.
 - ✓ 304 Not Modified: Tài nguyên không thay đổi kể từ lần truy vấn trước đó, client có thể sử dụng bản sao trong cache. Ứng dụng: Tối ưu hóa thời gian tải khi nội dung không thay đổi.

Nhóm mã trạng thái 4xx – Lỗi Client (Client Error)

- Mã trạng thái 4xx cho biết có lỗi từ phía client (do yêu cầu không hợp lệ hoặc không có quyền truy cập).
- Các mã trạng thái phổ biến:
 - ✓ 400 Bad Request: Yêu cầu không hợp lệ (có thể do cú pháp yêu cầu sai hoặc dữ liệu không hợp lệ). Ví dụ: Thiếu trường bắt buộc khi gửi form đăng ký.
 - ✓ 401 Unauthorized: Yêu cầu chưa được xác thực (cần đăng nhập hoặc token hợp lệ). Ví dụ: Truy cập tài nguyên bảo mật mà không có token.

Nhóm mã trạng thái 4xx – Lỗi Client (Client Error)

- Các mã trạng thái phổ biến:
 - ✓ 403 Forbidden: Truy cập bị từ chối, mặc dù người dùng đã xác thực. Ví dụ: Người dùng không có quyền truy cập trang quản trị.
 - ✓ 404 Not Found: Tài nguyên không tồn tại. Ví dụ: Yêu cầu đường dẫn đến một trang không tồn tại trong hệ thống.
 - ✓ 429 Too Many Requests: Người dùng đã gửi quá nhiều yêu cầu trong một khoảng thời gian ngắn. Ứng dụng: Giới hạn tốc độ (rate limiting) để tránh tấn công DDoS.

Nhóm mã trạng thái 5xx - Lỗi Server (Server Error)

- Mã trạng thái 5xx cho biết máy chủ gặp lỗi và không thể xử lý yêu cầu.
- Các mã trạng thái phổ biến:
 - ✓ 500 Internal Server Error: Lỗi tổng quát khi máy chủ gặp vấn đề trong quá trình xử lý. Ví dụ: Lỗi cú pháp trong mã hoặc lỗi cơ sở dữ liệu trên máy chủ.
 - ✓ 502 Bad Gateway: Máy chủ nhận được phản hồi không hợp lệ từ máy chủ khác. Ví dụ: Hệ thống proxy hoặc load balancer gặp lỗi khi lấy dữ liệu từ máy chủ khác.

Nhóm mã trạng thái 5xx - Lỗi Server (Server Error)

- Các mã trạng thái phổ biến:
 - ✓ 503 Service Unavailable: Máy chủ tạm thời không khả dụng (do quá tải hoặc bảo trì). Ví dụ: Hệ thống bảo trì định kỳ và tạm dừng dịch vụ.
 - ✓ 504 Gateway Timeout: Máy chủ không nhận được phản hồi kịp thời từ máy chủ khác. Ví dụ: Quá thời gian chờ khi truy cập tài nguyên từ máy chủ API phụ thuộc.

Các nhóm mã trạng thái

- 2xx – Thành công
- 3xx – Điều hướng
- 4xx – Lỗi Client
- 5xx – Lỗi Server

- Tạo ứng dụng nhỏ với các endpoint trả về mã trạng thái cụ thể. Chẳng hạn:
 - ✓ Endpoint /success trả về 200 OK.
 - ✓ Endpoint /error trả về 500 Internal Server Error.
 - ✓ Endpoint /notfound trả về 404 Not Found.
- Yêu cầu viết mã xử lý và trả về các mã trạng thái này theo ngữ cảnh.

- Phản hồi chính xác mã trạng thái giúp người dùng và developer hiểu rõ trạng thái của yêu cầu.
- Các mã trạng thái giúp cải thiện trải nghiệm người dùng và giúp hệ thống phát triển dễ duy trì.

- Webservice hay còn được gọi là dịch vụ web, nó bao gồm các loại giao thức, tiêu chuẩn mở thường được sử dụng trong việc kết nối dữ liệu giữa các phần mềm ứng dụng với nhau
- Webservice hoạt động trên các giao thức chuẩn như HTTP/HTTPS và sử dụng định dạng dữ liệu như XML, JSON để truyền tải thông tin
- Webservice thường được sử dụng để xây dựng các hệ thống phân tán và tạo điều kiện cho các ứng dụng chia sẻ dữ liệu, tài nguyên một cách dễ dàng và có thể mở rộng.
- Web API: Đây là khái niệm rộng nhất, bao gồm tất cả các giao diện lập trình ứng dụng sử dụng HTTP để giao tiếp với một máy chủ web. Nó như một chiếc ô bao trùm các loại API khác nhau.

- **Độc lập ngôn ngữ:** Các ứng dụng sử dụng các ngôn ngữ khác nhau (Java, .NET, Python) đều có thể giao tiếp thông qua Webservice.
- **Dựa trên giao thức chuẩn:** Sử dụng các giao thức như HTTP, HTTPS để truyền tải dữ liệu.
- **Độc lập nền tảng:** Chỉ cần định dạng dữ liệu được đồng thuận (ví dụ: XML, JSON).
- **Khả năng mở rộng cao:** Thích hợp cho các hệ thống lớn với nhiều ứng dụng khác nhau.

1. SOAP (Simple Object Access Protocol)

- **SOAP** là một giao thức dựa trên XML cho việc trao đổi thông điệp giữa các ứng dụng qua các giao thức mạng (HTTP, SMTP, TCP).
- SOAP sử dụng XML để định dạng các thông điệp, cung cấp các tính năng bảo mật mạnh mẽ và thường được sử dụng trong các môi trường doanh nghiệp lớn
- SOAP cung cấp một cấu trúc chính xác cho các yêu cầu và phản hồi, giúp việc giao tiếp đáng tin cậy và an toàn hơn

1. SOAP (Simple Object Access Protocol)

- **Đặc điểm:**

- ✓ Được thiết kế để có khả năng mở rộng và tương thích cao.
- ✓ Hỗ trợ các giao thức bảo mật và giao dịch phức tạp.
- ✓ Có thể chạy trên bất kỳ giao thức truyền thông nào, không chỉ HTTP.

1. SOAP (Simple Object Access Protocol)

- **Ưu điểm:**

- ✓ Được sử dụng trong các hệ thống yêu cầu độ bảo mật cao và tính giao dịch phức tạp, như ngân hàng, tài chính.
- ✓ Tính tương thích mạnh mẽ và dễ mở rộng.

- **Nhược điểm:**

- ✓ Phức tạp, yêu cầu cấu trúc chặt chẽ, gây khó khăn cho các ứng dụng nhỏ và yêu cầu đơn giản.
- ✓ Tốc độ xử lý chậm hơn do sử dụng XML

2. RESTful (Representational State Transfer)

- **RESTful API** là một phong cách kiến trúc webservice sử dụng các phương thức HTTP (GET, POST, PUT, DELETE) để thực hiện việc trao đổi dữ liệu.
- RESTful API thường sử dụng định dạng JSON hoặc XML để truyền dữ liệu và hoạt động theo mô hình tài nguyên (resources).
- **Sử dụng trong các hệ thống:** Mạng xã hội, thương mại điện tử, API công cộng (Google Maps, OpenWeather, ...).
- **RESTful API** thường được sử dụng để xây dựng các ứng dụng web hiện đại, ứng dụng di động và IoT.

2. RESTful (Representational State Transfer)

- **Đặc điểm:**

- ✓ Đơn giản và nhẹ hơn so với SOAP, không yêu cầu cấu trúc phức tạp.
- ✓ Dữ liệu có thể được gửi ở định dạng JSON, XML hoặc các định dạng khác

2. RESTful (Representational State Transfer)

- **Ưu điểm:**

- ✓ Thân thiện với người dùng và dễ tích hợp với các ứng dụng web hiện đại.
- ✓ Tốc độ xử lý nhanh hơn và dễ sử dụng hơn so với SOAP, đặc biệt với các ứng dụng đơn giản.
- ✓ Sử dụng trực tiếp các phương thức HTTP (GET, POST, PUT, DELETE), giúp giảm chi phí và tài nguyên.

2. RESTful (Representational State Transfer)

- **Nhược điểm:**
 - ✓ Không hỗ trợ nhiều tính năng như SOAP (ví dụ: bảo mật WS-Security, giao dịch phân tán).
 - ✓ Bảo mật phụ thuộc vào HTTPS và cơ chế xác thực bên ngoài.

So sánh SOAP API và RESTful API

Tiêu chí	SOAP API	RESTful API
Giao thức	Giao thức riêng dựa trên XML.	HTTP.
Định dạng dữ liệu	XML.	JSON, XML, HTML, Plain Text.
Độ phức tạp	Phức tạp, nặng.	Đơn giản, nhẹ hơn SOAP.
Bảo mật	WS-Security.	Phụ thuộc vào HTTPS.
Sử dụng phổ biến	Hệ thống tài chính, doanh nghiệp.	Web API, mobile apps, microservices.

Phân biệt

Tính năng	REST API	Web API	SOAP API
Kiến trúc	Không trạng thái, dựa trên tài nguyên	Đa dạng, có thể có hoặc không có trạng thái	Dựa trên giao thức
Định dạng dữ liệu	JSON, XML, văn bản thuần túy	Đa dạng, thường là XML hoặc JSON	XML
Bảo mật	Có thể sử dụng nhiều cơ chế (ví dụ: OAuth, basic auth)	Có thể sử dụng nhiều cơ chế	Có các tính năng bảo mật tích hợp
Độ phức tạp	Đơn giản hơn, dễ học và triển khai	Tùy thuộc vào API cụ thể	Phức tạp hơn, cần nhiều chi phí hơn
Hiệu suất	Nói chung nhanh hơn và hiệu quả hơn	Tùy thuộc vào triển khai	Có thể chậm hơn do chi phí xử lý XML

Debugging (gỡ lỗi) là một kỹ năng quan trọng để xác định và sửa chữa lỗi trong các ứng dụng web. Dưới đây là một số kỹ thuật và công cụ phổ biến để debug một ứng dụng web:

- Kỹ thuật Debug phía Client (Frontend)
- Kỹ thuật Debug phía Server (Backend)

1. Công cụ Developer Tools trong trình duyệt

Hầu hết các trình duyệt hiện đại (Chrome, Firefox, Edge) đều cung cấp công cụ DevTools mạnh mẽ:

- **Inspect Element:** Kiểm tra và chỉnh sửa trực tiếp HTML/CSS.
- **Console Tab:** Hiển thị lỗi JavaScript và cho phép thực hiện lệnh debug.
- **Network Tab:** Kiểm tra các yêu cầu HTTP/HTTPS (GET, POST, PUT, DELETE), thời gian phản hồi, và trạng thái.
- **Sources Tab:**
 - Đặt breakpoint trong mã JavaScript để dừng tại một dòng cụ thể.
 - Kiểm tra giá trị của biến trong runtime.
- **Performance Tab:** Phân tích hiệu năng và thời gian tải trang.
- **Application Tab:** Kiểm tra lưu trữ localStorage, sessionStorage, cookies và cache.

2. Sử dụng Console

- Console là một công cụ tích hợp trong trình duyệt, cho phép tương tác trực tiếp với JavaScript đang chạy trên trang web.
- Tại sao dùng Console?
 - ✓ In ra thông tin: Kiểm tra giá trị của biến, xem các hàm được gọi như thế nào, theo dõi luồng thực thi của code.
 - ✓ Gọi hàm: Thực thi các hàm trực tiếp trong console để kiểm tra kết quả.
 - ✓ Cập nhật DOM: Thay đổi nội dung HTML, CSS để xem ảnh hưởng trực quan

2. Sử dụng Console

- Các lệnh cơ bản:
 - ✓ `console.log()`: In ra thông tin
 - ✓ `console.warn()`: Hiển thị cảnh báo
 - ✓ `console.error()`: Hiển thị lỗi
 - ✓ `console.table()`: Hiển thị dữ liệu dưới dạng bảng
 - ✓ `console.time()`, `console.timeEnd()`: Đo thời gian thực thi

2. Sử dụng Console

- Cách sử dụng:
 - ✓ Mở console: Nhấn F12 (hoặc Ctrl+Shift+I trên Windows/Linux, Cmd+Option+I trên Mac) để mở công cụ dành cho nhà phát triển của trình duyệt, chọn tab Console.
 - ✓ Viết lệnh: Nhập trực tiếp các lệnh vào console và nhấn Enter.
 - ✓ Đặt breakpoint: Tạm dừng việc thực thi code tại một dòng cụ thể để kiểm tra giá trị các biến tại thời điểm đó.
 - ✓ Ví dụ:

```
let message = "Hello, world!";  
console.log(message); // In ra "Hello, world!" vào console
```

3. Debug với Source Maps

Nếu mã JavaScript đã được minified (rút gọn), hãy kích hoạt Source Maps trong DevTools để debug file gốc.

1. Sử dụng Log

- Log là quá trình ghi lại các sự kiện xảy ra trong quá trình thực thi của ứng dụng.
- Tại sao dùng Log?
 - ✓ Theo dõi quá trình: Hiểu rõ ứng dụng đang làm gì, các hàm được gọi, dữ liệu được xử lý như thế nào.
 - ✓ Phát hiện lỗi: Tìm ra các lỗi tiềm ẩn, ví dụ như lỗi cú pháp, lỗi logic, lỗi kết nối.
 - ✓ Phân tích hiệu năng: Đo thời gian thực thi của các đoạn code để tối ưu hóa hiệu năng.

1. Sử dụng Log

■ Các loại Log:

- ✓ Log thông tin: Ghi lại các sự kiện bình thường.
- ✓ Log cảnh báo: Ghi lại các sự kiện bất thường nhưng không gây ảnh hưởng đến hoạt động của ứng dụng.
- ✓ Log lỗi: Ghi lại các lỗi nghiêm trọng khiến ứng dụng không hoạt động đúng.

1. Sử dụng Log

- Cách sử dụng:
 - ✓ Thư viện Logging: Sử dụng các thư viện như winston (Node.js), log4j (Java) để tùy chỉnh cách thức ghi log.
 - ✓ Cấu hình: Cấu hình mức độ chi tiết của log, định dạng log, nơi lưu trữ log.
 - ✓ Viết log: Sử dụng các hàm cung cấp bởi thư viện để ghi log vào các file hoặc console.

1. Sử dụng Logger

Sử dụng các thư viện logging như:

- **Java:** Log4j, SLF4J.
- **.NET:** Serilog, NLog.
- **Node.js:** Winston, Bunyan.
- **Đặt mức độ log:** DEBUG, INFO, WARN, ERROR.

2. Debug với Breakpoints

Eclipse/IntelliJ IDEA (Java), Visual Studio (.NET), hoặc VS Code (Node.js/Python):

- Đặt breakpoint trong mã.
- Chạy ứng dụng trong chế độ Debug để dừng tại các dòng cần kiểm tra.

3. Kiểm tra Log Server

- Kiểm tra log file được ghi bởi ứng dụng.
- Đảm bảo ghi rõ lỗi cụ thể và trace stack.

4. Debug API Endpoint

- Sử dụng công cụ như **Postman** hoặc **cURL** để gửi yêu cầu tới API và kiểm tra phản hồi.
- Ví dụ với cURL:

```
curl -X POST -H "Content-Type: application/json" -d  
'{"key": "value"}' http://localhost:8080/api
```

1. Kiểm tra các giao tiếp Frontend - Backend

Sử dụng **Network Tab** trong DevTools để kiểm tra yêu cầu từ Frontend:

- Trạng thái HTTP (200 OK, 404 Not Found, 500 Internal Server Error).
- Dữ liệu gửi/nhận.

Debug toàn bộ ứng dụng (Full Stack Debugging)

2. Kiểm tra Middleware

Nếu sử dụng framework như **Express.js (Node.js)** hoặc **Spring Boot (Java)**, thêm log để kiểm tra trạng thái request trong middleware.

3. Kiểm tra Database

- Kiểm tra các query trong log hoặc sử dụng công cụ debug như:

MySQL Workbench, PgAdmin (PostgreSQL), SQL Profiler (SQL Server).

- Đảm bảo các câu truy vấn SQL không bị lỗi hoặc chậm.

1. Frontend

- **Chrome DevTools, Firefox DevTools.**
- **React Developer Tools** (nếu ứng dụng sử dụng ReactJS).
- **Vue.js DevTools, Redux DevTools.**

2. Backend

- **Postman:** Kiểm tra API.
- **Fiddler:** Phân tích lưu lượng HTTP.
- **Swagger UI:** Test REST API.
- IDE với debugger tích hợp (IntelliJ IDEA, VS Code, Eclipse).

3. Full Stack Debugging

- **Docker Logs:** Xem log khi sử dụng Docker containers.
- **ELK Stack (Elasticsearch, Logstash, Kibana):** Phân tích log.
- **Jaeger** hoặc **Zipkin:** Debug ứng dụng microservices.

- 1. Xác định lỗi:** Từ lỗi hiển thị trên giao diện (UI) hoặc log trên server.
- 2. Tái hiện lỗi:** Tìm cách tái hiện lỗi trong môi trường phát triển.
- 3. Phân tích lỗi:** Xem stack trace, trạng thái request/response, hoặc console log.
- 4. Sửa lỗi:** Thay đổi mã nguồn, cấu hình hoặc môi trường.
- 5. Kiểm tra lại:** Đảm bảo lỗi đã được sửa và không ảnh hưởng đến các chức năng khác.

Các lỗi thường gặp khi phát triển Web API và cách debug

1. Lỗi cú pháp
2. Lỗi logic
3. Lỗi liên quan đến cơ sở dữ liệu
4. Lỗi liên quan đến HTTP
5. Lỗi liên quan đến bảo mật
6. Lỗi hiệu năng

- Trình debug của IDE: Hầu hết các IDE đều tích hợp trình debug giúp đặt breakpoint, xem giá trị biến, theo dõi stack trace.
- Công cụ network của trình duyệt: Dùng để kiểm tra các request và response.
- Postman: Công cụ để gửi các request HTTP và kiểm tra response.
- Logger: Ghi lại các thông tin vào log để theo dõi quá trình thực thi của ứng dụng.

- **Web client** là thành phần phía người dùng (front-end) trong hệ thống client-server, chịu trách nhiệm gửi yêu cầu đến server và hiển thị phản hồi.
- Các **Web client** phổ biến bao gồm:
 - ✓ Trình duyệt web (Chrome, Firefox, Safari).
 - ✓ Ứng dụng mobile hoặc desktop kết nối với web server.

- Gửi yêu cầu HTTP(S) đến server (thông qua giao thức GET, POST, PUT, DELETE).
- Hiển thị nội dung trả về từ server dưới dạng HTML, CSS, JavaScript, JSON, v.v.

1. Cross-Site Scripting (XSS)

- **Mô tả:**

- Lỗ hổng này xảy ra khi ứng dụng không xử lý đúng dữ liệu đầu vào, cho phép kẻ tấn công nhúng mã độc (JavaScript) vào trang web.
- Mã độc này sẽ được thực thi trong trình duyệt của người dùng.

- **Hậu quả:**

- Đánh cắp thông tin cookie, session.
- Chuyển hướng người dùng đến trang độc hại.

- **Phòng ngừa:**

- Lọc và mã hóa dữ liệu đầu vào (input validation, output encoding).
- Sử dụng HTTP header như Content-Security-Policy (CSP)

2. Insecure Direct Object References (IDOR)

- **Mô tả:**

- Xảy ra khi ứng dụng không kiểm tra quyền truy cập, cho phép kẻ tấn công truy cập hoặc thay đổi dữ liệu nhạy cảm bằng cách sửa thông tin định danh (ID) trong URL hoặc request.

- **Hậu quả:**

- Lộ dữ liệu cá nhân.
- Truy cập trái phép vào tài nguyên của người dùng khác.

- **Phòng ngừa:**

- Kiểm tra quyền truy cập (authorization) ở server-side.
- Không dựa vào thông tin gửi từ client để xác thực quyền.

3. Broken Authentication:

- **Mô tả:**

- Các cơ chế xác thực không an toàn cho phép kẻ tấn công đánh cắp tài khoản.

- **Hậu quả:**

- Đánh cắp tài khoản, thực hiện các hành vi trái phép.

- **Phòng ngừa:**

- Sử dụng xác thực mạnh (mật khẩu phức tạp, 2FA).
- Bảo mật thông tin xác thực (hash mật khẩu).

4. SQL Injection

- **Mô tả:**

- Xảy ra khi ứng dụng không kiểm soát tốt dữ liệu đầu vào, cho phép kẻ tấn công chèn mã SQL độc hại để thao tác cơ sở dữ liệu.

- **Hậu quả:**

- Truy cập, sửa đổi hoặc xóa dữ liệu trái phép.
- Gây nguy hiểm nghiêm trọng cho hệ thống.

- **Phòng ngừa:**

- Sử dụng Prepared Statements (tham số hóa truy vấn SQL).
- Kiểm tra và lọc dữ liệu đầu vào.

5. Cross-Site Request Forgery (CSRF)

- **Mô tả:**

- Kẻ tấn công gửi yêu cầu giả mạo đến server từ trình duyệt của nạn nhân đã đăng nhập, dẫn đến hành động trái phép.

- **Hậu quả:**

- Thực hiện giao dịch trái phép (chuyển tiền, đổi mật khẩu).

- **Phòng ngừa:**

- Sử dụng token CSRF (CSRF token).
- Kiểm tra header Referer hoặc Origin.

6. Session Hijacking:

- **Mô tả:**

- Kẻ tấn công đánh cắp session ID của người dùng để giả mạo phiên làm việc.

- **Hậu quả:**

- Chiếm quyền truy cập vào tài khoản người dùng.

- **Phòng ngừa:**

- Sử dụng HTTPS để bảo mật truyền thông.
- Tạo session ID ngẫu nhiên và khó đoán.
- Hết hạn session ID sau một khoảng thời gian.

7. Session Fixation:

- **Mô tả:**

- Kẻ tấn công ép người dùng sử dụng session ID do chúng tạo ra trước, sau đó chiếm quyền phiên.

- **Hậu quả:**

- Kẻ tấn công có thể kiểm soát phiên của nạn nhân.

- **Phòng ngừa:**

- Tạo mới session ID sau khi người dùng đăng nhập.

8. Session Timeout

- **Mô tả:**

- Nếu không thiết lập timeout cho phiên, các tài khoản người dùng có thể bị lạm dụng sau khi họ rời khỏi thiết bị.

- **Hậu quả:**

- Phiên làm việc bị kẻ xấu lợi dụng.

- **Phòng ngừa:**

- Cấu hình thời gian timeout hợp lý cho session.

9. Lỗ hổng liên quan đến Session

- **Mô tả:**

- Bao gồm các vấn đề như lộ session ID, sử dụng session không an toàn.

- **Phòng ngừa:**

- Bảo mật session ID bằng cách mã hóa và lưu trữ an toàn.
- Kích hoạt bảo vệ session bằng HTTPOnly, Secure flags.

10. Lỗ hổng liên quan đến Cookie

- **Mô tả:**

- Cookie chứa thông tin quan trọng có thể bị đánh cắp qua XSS, hoặc bị sử dụng sai mục đích.

- **Phòng ngừa:**

- Kích hoạt các cờ bảo mật cho cookie:
 - **Secure:** Chỉ truyền cookie qua HTTPS.
 - **HttpOnly:** Ngăn mã JavaScript truy cập cookie.
 - **SameSite:** Ngăn các yêu cầu từ nguồn khác sử dụng cookie.

- **HTTPS** (HyperText Transfer Protocol Secure) là phiên bản bảo mật của giao thức HTTP. Nó bảo vệ dữ liệu được trao đổi giữa trình duyệt và server bằng cách mã hóa.
- **HTTPS** sử dụng giao thức mã hóa SSL/TLS để bảo mật kết nối.
- **Vai trò:**
 - ✓ Bảo vệ dữ liệu khỏi bị đánh cắp hoặc sửa đổi trong quá trình truyền tải.
 - ✓ Xác minh danh tính của server để người dùng biết họ đang giao tiếp với đúng server.

SSL/TLS là gì?

- **SSL (Secure Sockets Layer)** và **TLS (Transport Layer Security)** là các giao thức mật mã đảm bảo kết nối an toàn.
- **TLS** là phiên bản cải tiến của SSL, cung cấp bảo mật cao hơn và hiệu suất tốt hơn.
- Các thành phần bảo mật:
 - **Mã hóa (Encryption)**: Bảo mật dữ liệu truyền tải.
 - **Xác thực (Authentication)**: Đảm bảo danh tính của các bên.
 - **Toàn vẹn dữ liệu (Data Integrity)**: Phát hiện nếu dữ liệu bị sửa đổi trong quá trình truyền tải.

- **Chứng chỉ SSL (SSL Certificate)** là một tệp tin điện tử được cấp bởi các tổ chức xác thực (Certificate Authorities - CA) để xác thực danh tính của website và kích hoạt HTTPS.
- **Thành phần chính của chứng chỉ SSL:**
 - ✓ Tên miền hoặc máy chủ được bảo mật.
 - ✓ Tên tổ chức hoặc cá nhân sở hữu chứng chỉ.
 - ✓ Cơ quan cấp chứng chỉ (CA).
 - ✓ Ngày hết hạn của chứng chỉ.
 - ✓ Khóa công khai (Public Key).

1. Chứng chỉ miền (Domain Validation - DV)

- ✓ Chỉ xác minh quyền sở hữu tên miền.
- ✓ Phù hợp với các cá nhân hoặc website nhỏ.

2. Chứng chỉ tổ chức (Organization Validation - OV)

- ✓ Xác minh cả quyền sở hữu tên miền và thông tin tổ chức.
- ✓ Phù hợp với các công ty, tổ chức.

3. Chứng chỉ mở rộng (Extended Validation - EV)

- ✓ Xác minh sâu thông tin tổ chức và hiển thị tên tổ chức trên thanh trình duyệt.
- ✓ Phù hợp với các website thương mại điện tử hoặc tài chính.

1. Thành phần cơ bản

Thành phần liên quan

- **Client:** Trình duyệt hoặc ứng dụng gửi yêu cầu kết nối đến máy chủ.
- **Server:** Máy chủ chứa trang web hoặc API yêu cầu giao tiếp bảo mật.
- **Certificate Authority (CA):** Tổ chức cấp chứng chỉ SSL để xác thực danh tính của máy chủ.

Khóa và mã hóa

- **Khóa công khai (Public Key):** Được sử dụng để mã hóa dữ liệu, khóa này nằm trong chứng chỉ SSL của server.
- **Khóa riêng (Private Key):** Được server giữ bí mật và dùng để giải mã dữ liệu.
- **Session Key:** Khóa phiên được tạo ra trong quá trình handshake, dùng để mã hóa dữ liệu trong suốt phiên làm việc.

2. Quy trình chi tiết

Bước 1: Client Hello

Client (trình duyệt hoặc ứng dụng) gửi yêu cầu đến Server, bao gồm:

- ✓ Phiên bản giao thức SSL/TLS mà client hỗ trợ.
- ✓ Danh sách các thuật toán mã hóa (Cipher Suites) mà client có thể sử dụng.
- ✓ Một số ngẫu nhiên (Client Random) để tạo khóa sau này.

2. Quy trình chi tiết

Bước 2: Server Hello

Server phản hồi lại với:

- ✓ Phiên bản giao thức SSL/TLS mà server hỗ trợ.
- ✓ Thuật toán mã hóa đã được chọn từ danh sách của client.
- ✓ Một số ngẫu nhiên (Server Random).
- ✓ Chứng chỉ SSL của server (chứa Public Key, thông tin server và CA cấp chứng chỉ).

2. Quy trình chi tiết

Bước 3: Xác thực chứng chỉ

- **Client kiểm tra chứng chỉ SSL của Server:**
 - Chứng chỉ có được cấp bởi một **CA đáng tin cậy** không?
 - Chứng chỉ có hợp lệ (chưa hết hạn, không bị thu hồi) không?
 - Tên miền trong chứng chỉ có khớp với tên miền của server không?
- Nếu chứng chỉ hợp lệ, quá trình tiếp tục. Nếu không, client sẽ hiển thị cảnh báo bảo mật.

2. Quy trình chi tiết

Bước 4: Trao đổi khóa bảo mật

a. Sử dụng RSA (mã hóa không đối xứng):

1. **Client tạo Premaster Secret** (bí mật tiền mã hóa) – một chuỗi ngẫu nhiên.
2. Mã hóa **Premaster Secret** bằng Public Key (lấy từ chứng chỉ SSL của Server).
3. Gửi Premaster Secret đã mã hóa đến Server.
4. **Server** dùng Private Key của mình để giải mã và nhận Premaster Secret.

b. Sử dụng Diffie-Hellman (DH) (nếu áp dụng mã hóa khóa phiên):

Client và Server cùng nhau trao đổi các giá trị công khai và tính toán một khóa phiên dùng chung mà không cần truyền trực tiếp Premaster Secret.

2. Quy trình chi tiết

Bước 5: Tạo khóa phiên (Session Key)

- **Session Key** được cả Client và Server tính toán dựa trên:
 - Premaster Secret.
 - Client Random.
 - Server Random.
- **Session Key** sẽ được dùng để mã hóa toàn bộ dữ liệu truyền tải sau đó (mã hóa đối xứng, nhanh hơn mã hóa không đối xứng).

2. Quy trình chi tiết

Bước 6: Handshake hoàn tất

- Cả Client và Server gửi một thông báo "Finished" được mã hóa bằng Session Key để xác nhận mã hóa thành công.
- Sau khi xác nhận, phiên làm việc an toàn được bắt đầu.

2. Quy trình chi tiết

Bước 7: Truyền tải dữ liệu an toàn

- Tất cả dữ liệu truyền giữa Client và Server được mã hóa bằng Session Key.
- Quy trình mã hóa bảo vệ dữ liệu:
 - Tránh nghe lén (eavesdropping).
 - Phát hiện và ngăn chặn sửa đổi dữ liệu.

- HTTPS/SSL sử dụng sự kết hợp giữa mã hóa đối xứng và không đối xứng để bảo vệ dữ liệu:
 - ✓ **Mã hóa không đối xứng** (Public Key & Private Key): Được dùng trong Handshake.
 - ✓ **Mã hóa đối xứng** (Session Key): Được dùng để truyền tải dữ liệu sau Handshake.
- Quy trình này đảm bảo:
 - ✓ Dữ liệu được mã hóa và an toàn.
 - ✓ Danh tính của server được xác thực.
 - ✓ Dữ liệu không bị sửa đổi trong quá trình truyền tải



CMC UNIVERSITY

Aspire to Inspire the Digital World

THANK YOU

